qualityEvaluation MQTT/TLS/JSON

Interface Design Description

Service ID: *"qualityEvaluation"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `qualityEvaluation` service of the DeviceQoSEvaluator system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Service Operations **4**
3.1 operation filter . . . 4
3.2 operation sort . . . 4

4 Data Models **5**
4.1 struct MqttRequestTemplate . . . 5
4.2 struct MqttResponseTemplate . . . 5
4.3 struct ErrorMessage . . . 5
4.4 struct QoSEvaluationFilterResponse . . . 5
4.5 struct QoSEvaluationSortResponse . . . 5
4.6 Primitives . . . 6

5 Go and Rust implementation variants **7**
5.1 Go implementation . . . 7
5.2 Rust implementation . . . 7

6 References **8**

7 Revision History **9**
7.1 Amendments . . . 9
7.2 Quality Assurance . . . 9

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `qualityEvaluation` service interface of the DeviceQoSEvaluator system over MQTT, which provides quality-of-service evaluation of candidate providers from measurements of the devices hosting them (round-trip time, CPU load, memory use, network load).

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `qualityEvaluation` service interface

This document provides the Interface Design Description IDD to the *qualityEvaluation – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/deviceqosevaluator/qualityevaluation/`. The standalone deployment sets `mqtt.api.enabled=false`, so the MQTT interfaces are not registered there.

3 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`filter`	`arrowhead/deviceqosevaluator/qualityevaluation/filter`
`sort`	`arrowhead/deviceqosevaluator/qualityevaluation/sort`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

3.1 PUBLISH `arrowhead/deviceqosevaluator/qualityevaluation/filter`

Operation: filter
Output: QoSEvaluationFilterResponse

Returns the providers that meet the threshold of the weighted metrics, and the dropped ones.

3.2 PUBLISH `arrowhead/deviceqosevaluator/qualityevaluation/sort`

Operation: sort
Output: QoSEvaluationSortResponse

Returns the providers ordered by the weighted metrics, best first.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

4.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 3).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

4.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

4.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.4 struct QoSEvaluationFilterResponse

Result of a filter evaluation.

Object Field	Value Type	Description
"passedProviders"	Array<String>	Providers that meet the threshold.
"droppedProviders"	Array<String>	Providers that do not meet it.
"warnings"	Object<Array<String>>	Per provider, warnings such as missing measurements.

4.5 struct QoSEvaluationSortResponse

Result of a sort evaluation.

Object Field	Value Type	Description
"sortedProviders"	Array<String>	Providers ordered from best to worst.
"warnings"	Object<Array<String>>	Per provider, warnings such as missing measurements.

4.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-DeviceQoSEvaluator-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8472 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8472"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Measurement reload, weighted time-window means, threshold filtering and ranking; missing data rejected. Full Java SNMP/OID discovery, augmentation, normalization and scaling algorithm. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-DeviceQoSEvaluator-go/documentation/DeviceQoSEvaluator`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-DeviceQoSEvaluator-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8472 (Arrowhead default, role deviceqosevaluator). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Windowed weighted-mean filter/sort, measurement adapter. Java SNMP/OID pipeline and normalization. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/DeviceQoSEvaluator`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “qualityEvaluation – Service Description (DeviceQoSEvaluator),” Eclipse Arrowhead, SD 5.3.0, 2026, deviceQoSEvaluator-v5.3.0-qualityEvaluation-SD.pdf.

[6] ——, “qualityEvaluation HTTP/TLS/JSON – Interface Design Description (DeviceQoSEvaluator),” Eclipse Arrowhead, IDD 5.3.0, 2026, deviceQoSEvaluator-v5.3.0-qualityEvaluation-HTTP-IDD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	